

Introduction to Processor Architecture

Project Phase II: Pipelined RISC-V Processor Design

Spring 2026

Gaurav Reddy Yerreddi — 2024112006

Monishram Selvaraj — 2024102076

Neel Rajesh - 2024102007

1 Introduction

This report documents our implementation of a **5-stage Pipelined RISC-V Processor**, building upon the sequential processor developed in Phase I. The pipeline improves instruction throughput by decomposing execution into five concurrent stages. Our design addresses the key challenges of pipelining—data hazards, control hazards, and structural hazards—through forwarding logic, hazard detection, and pipeline flushing.

The processor supports the same subset of the RISC-V ISA as the sequential design.

2 The 5-Stage Pipeline

2.1 Pipeline Stages

The datapath is decomposed into the following five stages:

1. **Instruction Fetch (IF):** Fetches the instruction from Instruction Memory using the current PC and increments the PC to $PC + 4$ or branch target address.
2. **Instruction Decode (ID):** Decodes the fetched instruction, reads source operands from the Register File, generates the sign-extended immediate value, and produces all control signals.
3. **Execute (EX):** Performs the ALU operation (arithmetic, logical, comparison) and computes branch target addresses.
4. **Memory (MEM):** Accesses Data Memory for load and store instructions.
5. **Write Back (WB):** Writes the result (ALU output or memory read data) back to the Register File.

2.2 Pipeline Registers

Four pipeline register modules are instantiated between consecutive stages to hold both data and control signals. This section provides description of the IF/ID, ID/EX, EX/MEM, MEM/WB registers.

- **IF/ID Registers** — Store the fetched instruction and the value of $PC + 4$. These values are passed from the Instruction Fetch (IF) stage to the Instruction Decode (ID) stage so that the next stage can decode the instruction while the IF stage simultaneously fetches the next instruction.
- **ID/EX Registers** — Store decoded operands (`ReadData1`, `ReadData2`), the sign-extended immediate, register addresses (`rs1`, `rs2`, `rd`), `PC`, and all control signals (`RegWrite`, `MemRead`, `MemWrite`, `MemToReg`, `ALUSrc`, `ALUOp`, `Branch`). This register also supports flushing (NOP insertion for both stalls and branch flushes).
- **EX/MEM Registers** — Store the ALU result, the second operand (for store instructions), the zero/branch-condition flag, the destination register `rd`, and the propagated control signals (`RegWrite`, `MemRead`, `MemWrite`, `MemToReg`, `Branch`).
- **MEM/WB Registers** — Store the memory read data, the ALU result, the destination register `rd`, and write-back control signals (`RegWrite`, `MemToReg`).

All control signals propagate through the pipeline alongside data so that each stage can operate independently.

2.3 Pipeline Datapath Diagram

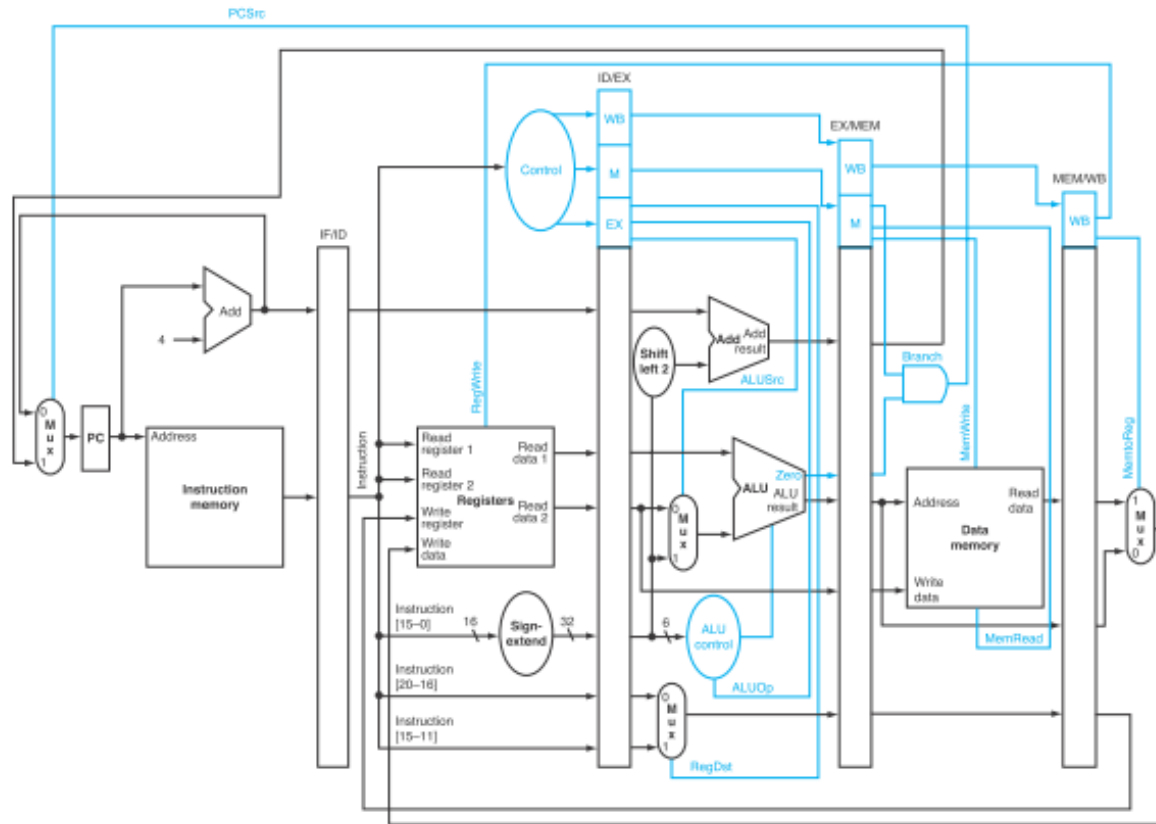


Figure 1: Complete pipelined datapath

3 Hazard Handling

3.1 Data Hazards

A **data hazard** occurs when an instruction in the pipeline depends on the result of a previous instruction that has not yet completed its write-back to the Register File. In a sequential (non-pipelined) processor this is never a problem because each instruction finishes entirely *before* the next one begins. However, in a pipelined processor, multiple instructions are *in flight* simultaneously, and a later instruction may attempt to *read* a register that an earlier instruction has not yet *written*. If left unhandled, the pipeline would read stale (old) values, producing incorrect results.

Definition. A data hazard exists between two instructions I_i (producer) and I_j (consumer), where I_i precedes I_j in program order, whenever I_j needs a value that I_i has not yet made available..

3.1.1 Types of Data Hazards

Data hazards are classified into three categories based on the *ordering* of read and write operations between two dependent instructions:

A RAW hazard occurs when instruction I_j tries to **read** a register that instruction I_i has not yet **written**.

```
add  x1, x2, x3    ←  $I_i$ : writes to x1
sub  x4, x1, x5    ←  $I_j$ : reads x1
```

Here, I_j (sub) needs the value of **x1** that I_i (add) produces. In the pipeline, I_j reaches the ID stage (where register reads occur) while I_i is only in the EX stage — the result has not yet been written back to the Register File.

RAW hazards are the **most common** and the **only** type of data hazard that occurs in the standard RISC-V 5-stage pipeline. This is because instructions always read in the ID stage (early) and write in the WB stage (late), so a subsequent instruction can attempt to read a value before the prior instruction writes it.

RAW hazards are resolved using two complementary techniques:

- **Data Forwarding (Bypassing):** The result is sent directly from a later pipeline stage (EX/MEM or MEM/WB) back to the ALU inputs, bypassing the Register File entirely. This eliminates the need to wait for the write-back stage.
- **Pipeline Stalling:** In the special case of a *load-use hazard* (where the producing instruction is a **lw** and the consuming instruction immediately follows), forwarding alone is insufficient because the data is not available until the end of the MEM stage. The pipeline must be stalled for one cycle to allow the load to complete, after which the value can be forwarded.

1. Write After Write (WAW) — Output Dependency

A WAW hazard occurs when two instructions both **write** to the **same** register, and the writes could complete out of program order, leaving the wrong value in the register.

```
add  x1, x2, x3    ←  $I_i$ : writes to x1
sub  x1, x4, x5    ←  $I_j$ : also writes to x1
```

In correct program order, **x1** should hold the result of I_j after both instructions complete. A WAW hazard would occur if I_i 's write happened *after* I_j 's write, overwriting the correct value with the stale one.

In our **5-stage in-order pipeline**, **WAW hazards do not occur**. Since instructions enter and leave the pipeline in strict program order, and all writes happen in the same stage (WB), I_i will always write before I_j . WAW hazards are a concern only in *out-of-order* processors where instructions may complete in a different order than they were issued. They are not applicable in this project.

2. Write After Read (WAR) — Anti-Dependency

A WAR hazard occurs when instruction I_j **writes** to a register that instruction I_i has not yet **read**, potentially overwriting the value before I_i can use it.

```
add  x3, x1, x2    ←  $I_i$ : reads x1
sub  x1, x4, x5    ←  $I_j$ : writes to x1
```

Here, I_i needs to read the *old* value of $x1$, but I_j might write a *new* value to $x1$ before I_i has had a chance to read it.

In our **5-stage in-order pipeline**, **WAR hazards do not occur**. Reads happen in the ID stage (stage 2) and writes happen in the WB stage (stage 5). Since I_i is ahead of I_j in the pipeline, I_i will always read *before* I_j writes. WAR hazards become relevant in *out-of-order* processors or pipelines with *early writes* and *late reads*. They are not applicable in this project.

3.1.2 RAW Hazard Sub-Cases in the 5-Stage Pipeline

Since RAW is the only hazard type that occurs in our design, it is important to understand the specific pipeline scenarios in which it manifests. Consider a producing instruction I_i that writes to register **rd**, followed by a consuming instruction I_j that reads **rd** via **rs1** or **rs2**:

Case 1: I_j is 1 instruction after I_i (EX Hazard).

When I_j is in the **EX** stage, I_i is in the **MEM** stage. The result is available at the output of the EX/MEM pipeline register and can be forwarded directly to the ALU input of I_j .

```
add  x1, x2, x3    — in MEM stage
sub  x4, x1, x5    — in EX stage (needs x1)
```

Resolution: Forward from **EX/MEM** register to the EX stage ALU input. No stall required.

Case 2: I_j is 2 instructions after I_i (MEM Hazard).

When I_j is in the **EX** stage, I_i is in the **WB** stage. The result is available at the output of the MEM/WB pipeline register and can be forwarded to the ALU input of I_j .

```
add  x1, x2, x3    — in WB stage
nop
sub  x4, x1, x5    — in EX stage (needs x1)
```

Resolution: Forward from **MEM/WB** register to the EX stage ALU input. No stall required.

Case 3: I_j is 3 or more instructions after I_i (No Hazard).

By the time I_j reaches the ID stage to read the Register File, I_i has already completed its WB stage and written the correct value. No forwarding or stalling is necessary.

Case 4: I_i is a Load instruction and I_j is 1 instruction after I_i (Load-Use Hazard).

This is a special and critical sub-case of the EX hazard. When I_i is a **lw** instruction, its result is not available until the *end* of the MEM stage — one cycle later than a regular ALU instruction. Therefore, when I_j is in the EX stage and needs the loaded value, I_i is in the MEM stage but the data has not yet been read from memory.

lw x1, 0(x2) — in MEM stage (data not yet available)
 add x3, x1, x4 — in EX stage (needs x1)

Resolution: Forwarding alone **cannot** resolve this because the data physically does not exist yet. The pipeline must be **stalled for one cycle**. After the stall, I_i has completed its MEM stage and the loaded value can be forwarded from the MEM/WB register to I_j in the EX stage.

Key Observation. The forwarding unit eliminates most RAW hazards without any performance penalty. The *only* case that incurs a one-cycle stall is the load-use hazard (Case 4), which is detected by the Hazard Detection Unit.

3.2 Data Forwarding Unit

Data hazards arise when an instruction in the EX stage requires a result that has not yet been written back. Our **Forwarding Unit** resolves these by bypassing data from later pipeline stages to the ALU inputs.

3.2.1 Forwarding Conditions

1. **EX Hazard (MEM $\rightarrow$ EX):** The result from the EX/MEM pipeline register is forwarded when:

- EX/MEM.RegWrite == 1
- EX/MEM.rd != 0
- EX/MEM.rd == ID/EX.rs1 or EX/MEM.rd == ID/EX.rs2

2. **MEM Hazard (WB $\rightarrow$ EX):** The result from the MEM/WB pipeline register is forwarded when:

- MEM/WB.RegWrite == 1
- MEM/WB.rd != 0
- No EX hazard is active for the same source register
- MEM/WB.rd == ID/EX.rs1 or MEM/WB.rd == ID/EX.rs2

3.2.2 Forwarding Mux Control

The forwarding unit outputs two 2-bit signals, **ForwardA** and **ForwardB**, selecting the ALU operand source:

Forward Signal	Value	Source
00	0	ID/EX register (no forwarding)
10	2	EX/MEM pipeline register (EX hazard)
01	1	MEM/WB pipeline register (MEM hazard)

Table 1: Forwarding mux control signal encoding.

3.2.3 Forwarding Unit Verilog Code

Listing 1: forwardingunit.v

```

1 module ForwardingUnit(
2     input  EX_MEM_RegWrite,
3     input  MEM_WB_RegWrite,
4
5     input  [4:0] EX_MEM_Rd,
6     input  [4:0] MEM_WB_Rd,
7
8     input  [4:0] ID_EX_Rs1,
9     input  [4:0] ID_EX_Rs2,
10
11     output reg [1:0] ForwardA,
12     output reg [1:0] ForwardB
13 );
14
15 always @(*) begin
16
17     ForwardA = 2'b00;
18     ForwardB = 2'b00;
19
20     // ForwardA
21     if (EX_MEM_RegWrite && (EX_MEM_Rd != 0) && (EX_MEM_Rd ==
22         ID_EX_Rs1))
23         ForwardA = 2'b10;
24     else if (MEM_WB_RegWrite && (MEM_WB_Rd != 0) &&
25         !(EX_MEM_RegWrite && (EX_MEM_Rd != 0) && (EX_MEM_Rd
26             == ID_EX_Rs1)) &&
27         (MEM_WB_Rd == ID_EX_Rs1))
28         ForwardA = 2'b01;
29
30     // ForwardB
31     if (EX_MEM_RegWrite && (EX_MEM_Rd != 0) && (EX_MEM_Rd ==
32         ID_EX_Rs2))
33         ForwardB = 2'b10;
34     else if (MEM_WB_RegWrite && (MEM_WB_Rd != 0) &&
35         !(EX_MEM_RegWrite && (EX_MEM_Rd != 0) && (EX_MEM_Rd
36             == ID_EX_Rs2)) &&
37         (MEM_WB_Rd == ID_EX_Rs2))
38         ForwardB = 2'b01;
39
40 end

```

3.3 Hazard Detection Unit

Forwarding alone cannot resolve **load-use hazards**, where a load instruction is immediately followed by an instruction that uses the loaded value. In this case, we must **stall** the pipeline for one cycle.

3.3.1 Stall Condition

A stall is inserted when:

- `ID/EX.MemRead == 1` (current instruction in EX is a load)
- `ID/EX.rd == IF/ID.rs1 or ID/EX.rd == IF/ID.rs2`

3.3.2 Stall Mechanism

When a load-use hazard is detected:

1. **PC Write Disable:** The PC register is prevented from updating.
2. **IF/ID Write Disable:** The IF/ID pipeline register holds its current value.
3. **Control Signal Flush:** All control signals in the ID/EX register are set to zero (NOP bubble).

3.3.3 Hazard Detection Unit Verilog Code

Listing 2: Hazard Detection Unit

```

1 module hazard_detection_unit(
2     input ID_EX_MemRead,
3     input [4:0] ID_EX_Rd,
4     input [4:0] IF_ID_Rs1,
5     input [4:0] IF_ID_Rs2,
6
7     output reg PCWrite,
8     output reg IF_ID_Write,
9     output reg stall
10 );
11
12 always @(*) begin
13     if (ID_EX_MemRead && (ID_EX_Rd != 5'b0) &&
14         ((ID_EX_Rd == IF_ID_Rs1) || (ID_EX_Rd == IF_ID_Rs2)))
15         begin
16             PCWrite = 0;
17             IF_ID_Write = 0;
18             stall = 1;
19         end
20     else begin
21         PCWrite = 1;
22         IF_ID_Write = 1;
23         stall = 0;
24     end
25 end
26 endmodule

```


3.4 Control Hazards

Our design uses a **static branch prediction: always NOT TAKEN** strategy.

- The PC is always updated to $PC + 4$ (sequential fetch).
- Branch resolution occurs in the **EX stage**.
- If the branch is **actually taken**, the two instructions fetched after the branch (in IF and ID stages) are invalid and must be **flushed**.

3.4.1 Flush Mechanism

When a branch is taken:

1. **IF/ID Register Flush:** The instruction in the IF/ID register is replaced with a NOP.
2. **ID/EX Register Flush:** The instruction in the ID/EX register is replaced with a NOP.
3. **PC Update:** The PC is loaded with the computed branch target address from the EX stage.

4 Test Cases and GTKWaveform Analysis

This section presents the test cases used to verify the pipelined processor, along with annotated waveform screenshots.

4.1 Test Case 1: Data Forwarding — EX Hazard (MEM to EX)

Instructions:

```

1 addi x2,x0,5
2 addi x3,x0,7
3 add  x1,x2,x3
4 sub  x4,x1,x2

```

Expected Behavior: The forwarding unit detects that $EX/MEM.rd$ matches $ID/EX.rs1$ and forwards the ALU result. No stall is needed.



Figure 2: Waveform showing EX hazard forwarding. $ForwardA = 10$.

4.2 Test Case 2: Data Forwarding — MEM Hazard (WB to EX)

Instructions:

```

1 addi x2,x0,5
2 addi x3,x0,7
3 add  x1,x2,x3
4 nop
5 sub  x4,x1,x2

```

Expected Behavior: The forwarding unit forwards from MEM/WB register since the EX hazard condition is not met.

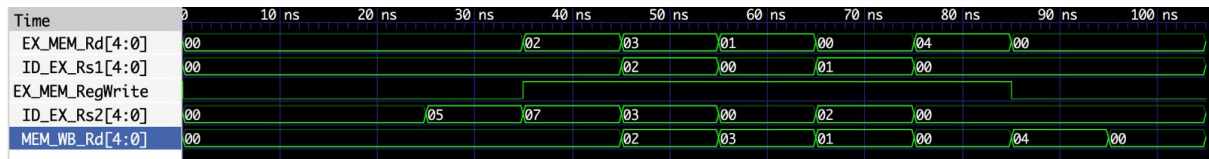


Figure 3: Waveform showing MEM hazard forwarding. ForwardA = 01.

4.3 Test Case 3: Load-Use Hazard (Stall + Forward)

Instructions:

```

1 addi x2, x0, 16
2 addi x4, x0, 3
3
4 lw    x1, 0(x2)
5 add   x3, x1, x4

```

Expected Behavior: The hazard detection unit inserts a one-cycle stall (bubble). After the stall, the loaded value is forwarded from MEM/WB to EX.



Figure 4: Waveform showing one-cycle stall for load-use hazard. PC and IF/ID are frozen; a NOP bubble is inserted.

4.4 Test Case 4: Control Hazard — Branch Taken (Flush)

Instructions:

```

1 addi x1, x0, 5
2 addi x2, x0, 5
3
4 beq  x1, x2, target
5

```

```

6 add x5, x6, x7
7 sub x8, x9, x10
8
9 target:
10 or x11, x12, x13

```

Expected Behavior: Since we predict NOT TAKEN, the two instructions after the branch are fetched. When the branch is resolved as taken in the EX stage, the IF/ID and ID/EX registers are flushed, and the PC is updated to `target`.

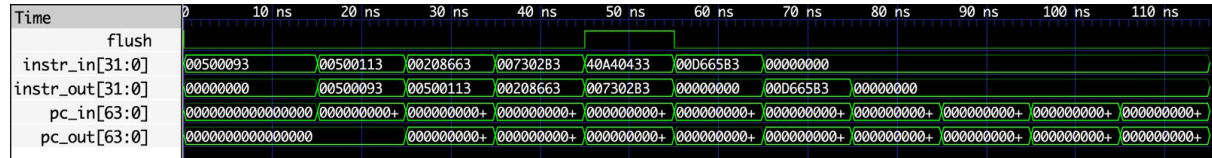


Figure 5: Waveform showing pipeline flush on a taken branch. Two instructions are squashed.

5 Example: instruction and given output

After executing the provided `instructions.txt` using our testbench (`iverilog pipe_tb.v`), the generated `register_file.txt` was compared against the golden reference. All 32 register values matched, and the clock cycle count was verified to contain no wasted cycles.

Instructions:

```

1 addi x1, x0, 8
2 addi x2, x0, 16
3 add x3, x1, x2
4 add x4, x2, x2
5
6 sd x4, 0(x3)
7
8 ld x5, 0(x3)
9 beq x5, x4, 12
10 addi x5, x0, 999
11 addi x4, x0, 888
12
13 add x6, x5, x4
14
15 addi x8, x0, 10
16 addi x8, x8, 10
17 add x0, x8, x8
18 sub x9, x8, x0
19
20 ld x10, 0(x3)
21 sd x10, 8(x3)
22
23 ld x11, 8(x3)
24 beq x11, x10, 8
25 addi x12, x0, 99

```

000000000000000000	000000000000000000
000000000000000008	000000000000000000
000000000000000010	000000000000000000
000000000000000018	000000000000000000
000000000000000020	000000000000000000
000000000000000020	000000000000000000
000000000000000040	000000000000000000
000000000000000000	000000000000000000
000000000000000014	000000000000000000
000000000000000014	000000000000000000
000000000000000020	000000000000000000
000000000000000020	000000000000000000
000000000000000000	000000000000000000
000000000000000000	000000000000000000
000000000000000003	000000000000000000
000000000000000003	40
000000000000000000	

values of its source registers by forwarding results from later pipeline stages (typically the EX/MEM or MEM/WB stages) instead of waiting for them to be written back to the register file. This ensures the branch condition is evaluated using the correct data without introducing additional pipeline stalls.

The forwarding logic checks whether the destination register of an instruction in a later pipeline stage matches either source register of the branch instruction. If a match is detected and the instruction writes to the register file, the forwarded value is used for the comparison instead of the value read from the register file.

Example Consider the following instruction sequence:

```
add  x1, x2, x3
beq  x1, x4, target
```

The `add` instruction produces the value of `x1`, but this result is not written back to the register file until the writeback stage. When the `beq` instruction reaches the execute stage to perform the comparison, the updated value of `x1` is still in the pipeline.

Without forwarding, the branch would compare the old value of `x1` (from the register file), potentially leading to an incorrect branch decision. With branch operand forwarding, the result of the `add` instruction is forwarded directly from the pipeline (e.g., EX/MEM stage) to the branch comparison logic, allowing the branch to evaluate the correct condition without stalling the pipeline.

Here is a test case showing its working

```
1 addi x1, x0, 5
2 addi x2, x0, 5
3 add  x3, x1, x2
4 beq  x3, x2, skip
5 addi x4, x0, 99
6 skip:
7 add  x5, x3, x2
```

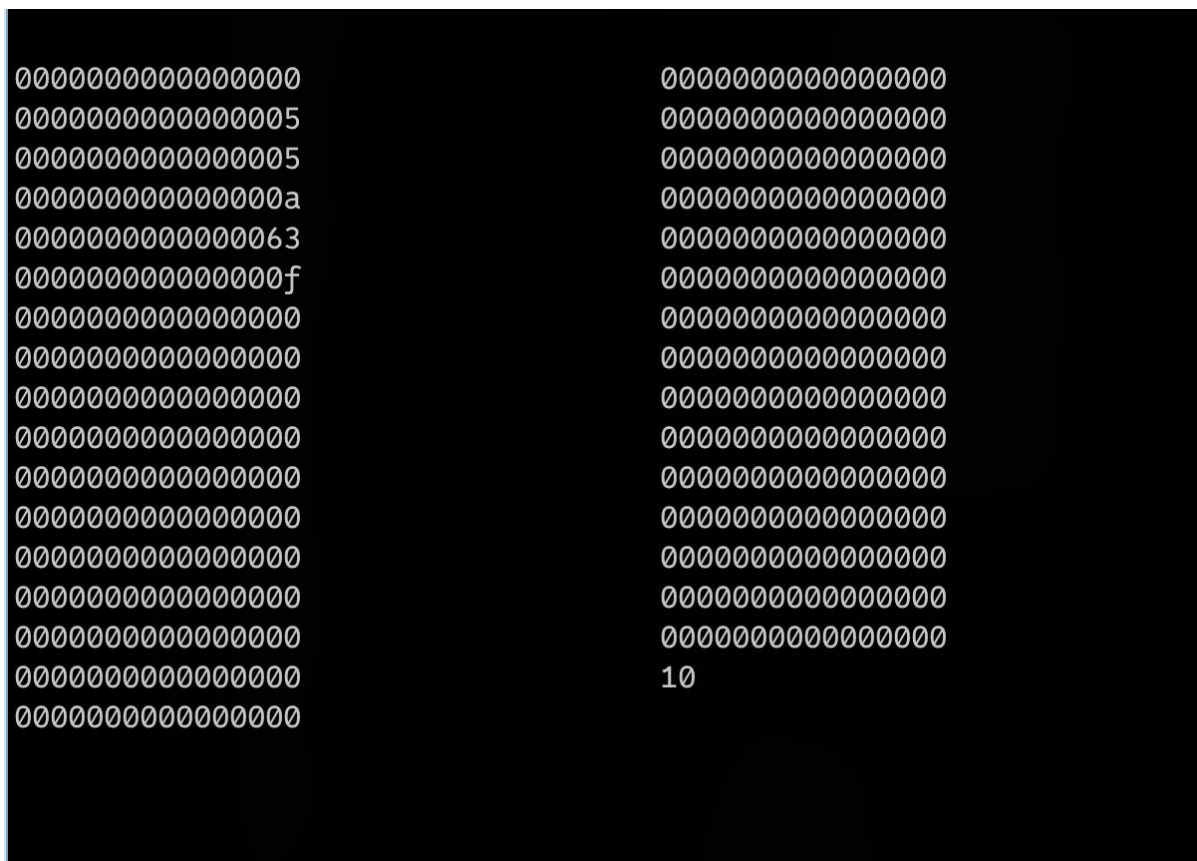


Figure 7: GTKwave output

7 Conclusion

We successfully implemented a 5-stage pipelined RISC-V processor that correctly handles data hazards (via forwarding and stalling) and control hazards (via flushing with static branch prediction: always not taken). The design was verified against the provided golden reference, with all register values and clock cycle counts matching the expected output.

8 Team Contributions

This section outlines the individual contributions of each team member towards the design, implementation, testing, and documentation of the pipelined RISC-V processor.

Team Member	Contributions
Gaurav Yerreddi Reddy (Roll No: 2024112006)	• Designed and implemented the 5-stage pipeline datapath, including the decomposition of the sequential processor into IF, ID, EX, MEM, and WB stages. • Implemented all four pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB) with proper propagation of data and control signals. • Integrated the control unit outputs into the pipeline register chain, ensuring all signals (RegWrite, MemRead, MemWrite, MemToReg, ALUSrc, ALUOp, Branch) propagate correctly through each stage. • Co-authored the report.
Monishram Selvaraj (Roll No: 2024102076)	• Implemented the control hazard handling logic, including the flush mechanism for the IF/ID and ID/EX registers when a branch is resolved as taken in the EX stage. • Designed and implemented the Hazard Detection Unit to handle load-use hazards by stalling the pipeline for one cycle (freezing PC and IF/ID register, inserting a NOP bubble into ID/EX). • Developed the testbench (<code>pipe_tb.v</code>) and created test cases covering all hazard scenarios: EX forwarding, MEM forwarding, load-use stalls, branch taken (flush), and branch not taken. • Performed GTKWave Analysis and co-authored the report
Neel Rajesh (Roll No: 2024102007)	• Implemented the forwarding unit logic for EX Hazard and MEM Hazard. • Implemented the forwarding mux logic at the ALU inputs to select between the register file output, EX/MEM forwarded value, and MEM/WB forwarded value. • Developed the testbench (<code>pipe_tb.v</code>) and created test cases covering all hazard scenarios: EX forwarding, MEM forwarding, load-use stalls, branch taken (flush), and branch not taken. • Performed waveform analysis using GTK-Wave, verified register file outputs against the golden reference, and authored the project report.

Table 2: Individual contributions of each team member.

All team members participated in collaborative debugging sessions, code reviews, and design discussions throughout the project. Final integration and verification were performed jointly to ensure correctness of the complete pipelined processor. Bonus was collaborated on by all three members of the team.